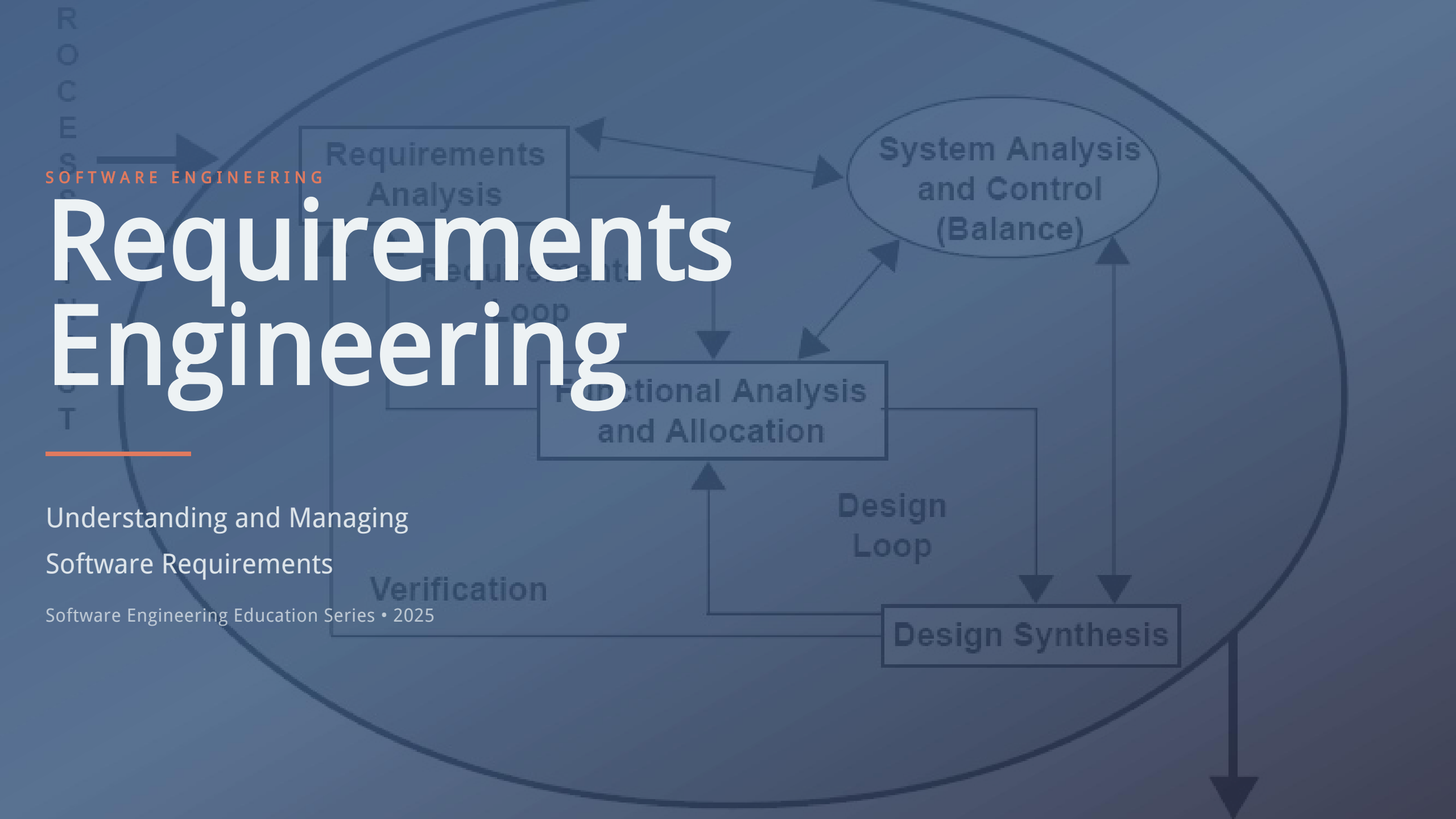
SOFTWARE ENGINEERING

Requirements Engineering

Understanding and Managing Software Requirements

Software Engineering Education Series • 2025



COMMUNICATION CHALLENGE

What Do You Think After Construction?



> "I know you believe you understood what you think I said, but I am not sure you realise that what you heard **is not what I meant**……"
>
> **The Critical Challenge of Requirements Communication**

This famous quote perfectly captures why requirements engineering is **so challenging** and why we need structured approaches.

Miscommunication in requirements leads to **project failure**, wasted resources, and stakeholder dissatisfaction.

FOUNDATIONAL CONCEPTS

What is a Requirement?

Requirement

A **contractual condition and/or capability** (external customer) or **industrial constraint** (system engineering group, marketing, etc.)

> **Key Principle:** States WHAT the system must do, not HOW it does it.

Specification

A **list of technical requirements** that define the system's capabilities, constraints, and characteristics in detail.

> **Essential for:** Ensuring development coherence and stakeholder alignment.

Requirement Traceability

Necessary to ensure development coherence throughout the project lifecycle. Traceability links requirements to design, implementation, and testing.

SwReq_1	SwReq_2	...	SwReq_X
The item shall...	...	...	...

CLASSIFICATION

Types of System Requirements

1 Functional Technical Requirements

Capabilities
Core functions the system must perform

Dynamic Behaviour
How the system responds and operates

Data Manipulation
Processing, storage, and retrieval

2 Non-Functional Technical Requirements

Operational Security	Safety	Availability	Reliability
Maintainability	Ergonomics	Performance	Constraints

3 Non-Technical Requirements

Contractual Milestones
Delivery schedules, payment terms, and project checkpoints

Required Methods & Techniques
Standards, tools, and processes to be used

QUALITY ATTRIBUTES

Characteristics of Excellent Requirements

Complete

Contains all necessary information for developers to implement correctly without needing additional clarification.

Correct

Accurately represents stakeholder needs and system objectives. Free from factual errors and misinterpretations.

Feasible

Achievable within available technology, budget, time constraints, and organizational capabilities.

Necessary

Essential for meeting business objectives and stakeholder needs. Not a 'nice-to-have' feature without justification.

Implementation Independent

Specifies WHAT is needed without dictating HOW to implement it, allowing design flexibility.

Prioritized

Ranked by importance and urgency to guide development sequence and resource allocation decisions.

Unambiguous

Clear, concise, and leaves no room for multiple interpretations. Uses precise language and defined terms.

Verifiable

Can be objectively tested and measured to determine if the requirement has been successfully implemented.

Right Level of Detail

Appropriate granularity for the intended audience and project phase—neither too vague nor overly specific.

Characteristics of Excellent Specifications

Complete

Includes all necessary requirements, definitions, and references. Nothing is missing that would prevent correct implementation.

Consistent

No internal contradictions or conflicting requirements. All terms and definitions are used uniformly throughout.

Concise

Clear, to the point, and free from unnecessary repetition or verbose explanations. Easy to read and understand.

Modifiable

Structured and organized to facilitate updates and changes. Can evolve with the project while maintaining integrity.

Traceable

Each requirement can be traced to its origin, implementation, and testing. Supports impact analysis of changes.

The Gold Standard

An excellent specification serves as the **foundation for successful software development**, providing clarity, consistency, and completeness that enables effective implementation and validation.

> "A specification that lacks these characteristics leads to misinterpretation, rework, and project failure."

CRITICAL IMPORTANCE

Why Requirements Engineering is Important

Communication Foundation

A software process action starting communication that continues into modeling activity. Requirements engineering establishes the initial dialogue between stakeholders and development teams.

Impact: Sets the tone for entire project collaboration.

Bridge to Design & Construction

The critical bridge between problem understanding and solution creation. Well-engineered requirements translate stakeholder needs into actionable development guidance.

Impact: Determines architecture quality and implementation success.

Understanding Needs

To understand the needs of process, project, product and people. Deep comprehension of stakeholder objectives, constraints, and success criteria is essential.

Impact: Ensures delivered system actually meets real user needs.

Key Success Factor

A key problem leading to failed software projects. Poor requirements engineering is consistently identified as a primary cause of project failure, budget overruns, and stakeholder dissatisfaction.

Impact: Poor requirements are a leading cause of project failure.

Requirements engineering is not optional—it's essential for success.

THE PROCESS

Requirements Engineering Tasks

Part 1: Foundation Activities

01 Inception

Ask a set of questions that establish the basic understanding of the problem and identify key stakeholders.

- Problem understanding
- People who want solution
- Nature of desired solution
- Communication effectiveness

02 Elicitation

Elicit requirements from all stakeholders through structured and unstructured interactions.

Goal: Extract needs, expectations, and constraints from everyone with a vested interest.

Methods: Interviews, workshops, observation, surveys, brainstorming sessions.

03 Elaboration

Create an analysis model that identifies data, function, and behavioral requirements.

Refinement: Transform raw requirements into structured analysis models.

Output: Use-cases, class diagrams, data flow diagrams, state models.

04 Negotiation

Agree on a deliverable system that is realistic for developers and valuable for customers. Resolve conflicts and establish priorities.

THE PROCESS

Requirements Engineering Tasks - Part 2

Specification • Validation • Management

05 Specification

Can be any one (or more) of the following formats, depending on project needs and stakeholder preferences:

- **A written document** - traditional requirements document
- **A set of models** - visual representation using UML/SysML
- **A formal mathematical** - precise mathematical notation
- **A collection of user scenarios** - use-cases and stories
- **A prototype** - working model or simulation

06 Validation

A review mechanism that looks for quality issues in the requirements and specifications:

Errors in content or interpretation - factual mistakes or misunderstandings

Areas where clarification may be required - ambiguous or unclear sections

Missing information - gaps in requirements coverage

Inconsistencies - conflicting or contradictory requirements

Conflicting or unrealistic requirements - unachievable objectives

07 Requirements Management

A set of activities that help the project team identify, control, and track requirements and changes throughout the project lifecycle.

Continuous Process: Not a one-time activity, but ongoing throughout development and maintenance.

- Identify requirements and changes
- Control and manage changes
- Track status and traceability

TASK 01: INCEPTION

Inception: Getting Started

Establishing the Foundation for Requirements Engineering

Identify Stakeholders

The first critical step is identifying all people who have an interest in the system. Go beyond obvious stakeholders to discover hidden or indirect stakeholders.

> **Key Question:** "Who else do you think I should talk to?"

Recognize Multiple Points of View

Different stakeholders have different perspectives, priorities, and concerns about the system. Acknowledge and document these varying viewpoints.

End Users	Management	IT Staff

Work Toward Collaboration

Foster collaborative relationships among stakeholders and between customers and developers. Build trust and open communication channels.

The First Questions

Who is behind the request?

Identify the project sponsor and decision-makers

Who will use the solution?

Define primary and secondary user groups

What will be the economic benefit?

Understand business value and ROI expectations

Is there another source?

Consider buy vs. build vs. outsource options

TASK 02: ELICITATION

Eliciting Requirements: Process

Structured Approach to Gathering Stakeholder Needs

Collaborative Meetings

Meetings are conducted and attended by both software engineers and customers to ensure mutual understanding.

Rules & Preparation

Rules for preparation and participation are established to ensure productive sessions with clear expectations.

Structured Agenda

An agenda is suggested to guide discussion, cover key topics, and maintain focus and momentum.

Facilitator Role

A "facilitator" controls the meeting to ensure productive discussion and equal participation.

- Customer
- Developer
- Outsider

Definition Mechanism

A "definition mechanism" is used to capture requirements in real-time during discussions.

- Worksheets
- Flip Charts
- Stickers
- E-Board
- Chat Room

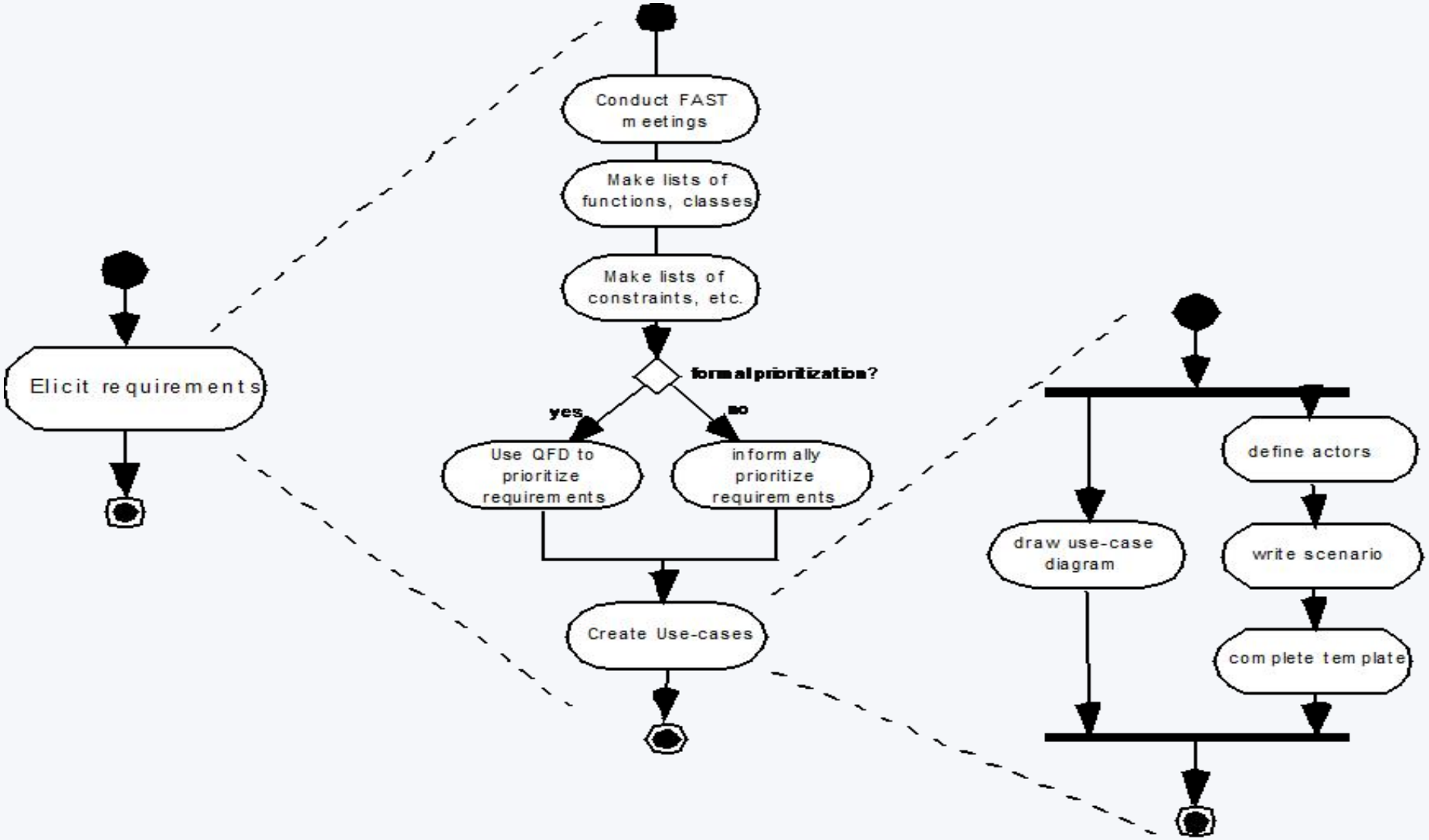
The Goal

1. **Identify** the problem
2. **Propose** solution elements
3. **Negotiate** different approaches
4. **Specify** preliminary requirements

VISUAL ELICITATION

Eliciting Requirements

Interactive and Collaborative Methods for Effective Requirements Discovery



QFD METHODOLOGY

Quality Function Deployment

Systematic Approach to Requirements Prioritization

1 Normal Requirements

Explicitly stated by the customer and easily understood. These are the requirements customers articulate when asked what they want.

Example: "The system must process 100 transactions per second."

2 Expected Requirements

So fundamental that customers assume they exist and don't bother to state them explicitly. These are implicit requirements.

Example: "The system should be available during business hours." (Unspoken)

3 Exciting Requirements

Beyond customer expectations and provide delight when present. Customers wouldn't think to ask for these, but they create competitive advantage.

Example: Predictive analytics that suggest optimizations.

Function Deployment

Determines the "value" (as perceived by the customer) of each function required of the system.

Maps system functions to customer needs and business objectives.

Value Analysis

Determines the relative priority of requirements based on customer value, implementation cost, and strategic importance.

Ensures high-value features are delivered first within budget constraints.

QFD Benefit: Transforms qualitative customer desires into quantitative technical specifications, ensuring development efforts deliver maximum customer value.

QFD METHODOLOGY

Quality Function Deployment

Systematic Approach to Requirements Prioritization

用户需求 (WHAT)	重要度	类型	一键挂号流程	号源同步接口	支付集成	退改签功能	消息推送	响应≤2秒	加密合规	可用性99.9%	权重
挂号操作简单快捷	5	期望型	9	3	3	3	1	9	1	1	132
号源准确不挂错	5	基本型	3	9	1	1	1	3	3	9	150
支付安全顺畅	5	基本型	1	1	9	1	1	3	9	3	140
退号退款方便	4	期望型	3	3	3	9	1	3	1	3	104
就诊提醒及时	4	期望型	1	1	1	1	9	1	1	3	72
老人也会用	4	基本型	9	1	1	3	3	3	1	1	88
隐私数据保护	5	基本型	1	1	1	1	1	1	9	3	90
高峰期不卡顿	5	基本型	1	3	3	1	1	9	1	9	140
检查报告查询	3	兴奋型	3	3	1	1	3	3	3	1	54
家人代挂号	3	兴奋型	3	1	3	1	1	1	9	1	60
相对权重			14%	11%	11%	8%	9%	16%	16%	15%	

DELIVERABLES

Elicitation Work Products

Key Outputs from Requirements Elicitation Activities

Statement of Need and Feasibility

High-level justification for the project, documenting business need, problem statement, and preliminary assessment of technical and economic feasibility.

Bounded Statement of Scope

Clear definition of system boundaries, what's in and out of scope, major functions, and constraints. Prevents scope creep.

List of Stakeholders

Comprehensive list of customers, users, and other stakeholders who participated, including roles, responsibilities, and contact information.

Description of Technical Environment

Overview of the technical context, including existing systems, platforms, integration points, and technical constraints.

List of Requirements & Constraints

Organized list of requirements by function, including functional, non-functional, and domain constraints that apply to each.

Set of Usage Scenarios

Collection of use-cases and scenarios describing system usage under different conditions, providing insight into user workflows.

Any Prototypes Developed

Working models or simulations developed to better define requirements, validate concepts, and reduce uncertainty.

TASK 03: ELABORATION

Elaboration: Building the Analysis Model

Creating Structured Representations of Requirements

Scenario-Based Elements

Functional
Processing narratives for software functions

Use-Case
Actor-system interaction descriptions

Purpose: Describe system behavior from user perspective.

Class-Based Elements

Class Diagram
(Implied by scenarios)

Purpose: Represent static structure and data relationships.

Behavioral Elements

State Diagram
Seq Diagram

Purpose: Show system states, transitions, and responses.

Flow-Oriented Elements

Data Flow Diagram

Purpose: Illustrate data movement and transformation.

Integrated Analysis Model

These elements work together to provide **multiple perspectives** on system requirements, ensuring comprehensive coverage and deep understanding.

BEHAVIORAL MODELING

Use-Cases Part I

Introduction to User Scenarios

What are Use-Cases?

A collection of user scenarios that describe the thread of usage of a system. Each scenario represents a specific interaction between an actor and the system to achieve a goal.

- User-Focused
- Goal-Oriented
- Story-Based

What is an Actor?

An **"actor"** is a person or device that interacts with the software in some way.

- **Person:** End user, administrator
- **Device:** Sensor, external system

Point of View

Each scenario is described from the **actor's point of view**, not the system's perspective.

> "What does the actor want to accomplish and why?"

Key Insight: Use-cases bridge the communication gap between technical and non-technical stakeholders.

BEHAVIORAL MODELING

Use-Cases Part II: Key Questions

Essential Questions Every Use-Case Scenario Should Answer

1 Who are the actors?

Identify the **primary actor** (initiates the interaction) and **secondary actors** (support the interaction).

2 What are the actor's goals?

Understand **what the actor wants to accomplish** and their motivation for using the system.

3 What preconditions should exist?

Define **what must be true before the story begins**—system state, actor conditions, and environmental factors.

4 What main tasks are performed?

Document the **step-by-step interaction** between actor and system to achieve the primary goal.

5 What exceptions might occur?

Consider **alternative flows and error conditions** that could happen during the interaction.

6 What variations are possible?

Identify **variations in the actor's interaction**—different ways to achieve the same goal.

7 What info is acquired/produced?

Specify **what system information the actor will acquire, produce, or change** during the interaction.

8 Must actor inform about changes?

Determine if **the actor must inform the system about changes in the external environment**.

9 What information is desired?

Clarify **what information the actor desires from the system** to support their decision-making.

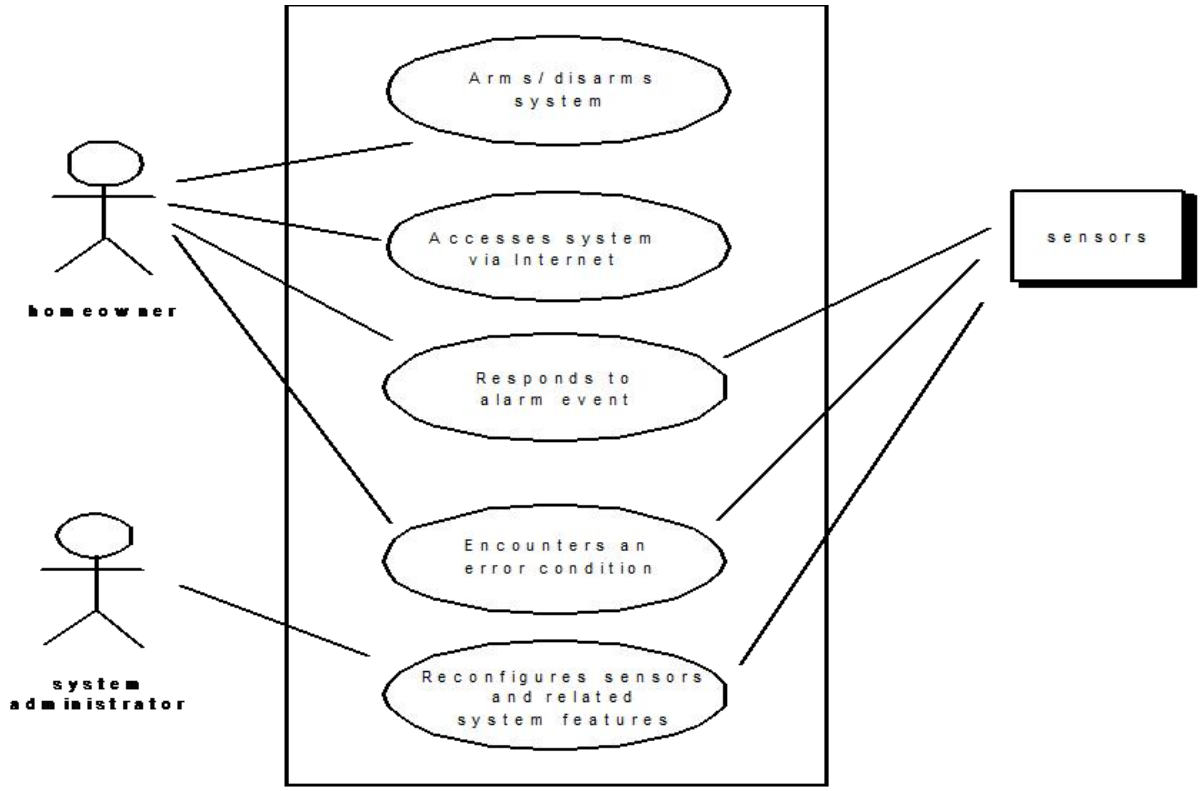
10 Wish to be informed of changes?

Ask if **the actor wishes to be informed about unexpected changes** or system events.

VISUAL REPRESENTATION

Use-Cases Diagram

High-Level View of System Boundaries, Actors, and Functionality



What is a Use-Case Diagram?

A **visual representation** showing system boundaries, actors, use cases, and their relationships.

Key Components

Actors
External entities (people or systems)

Use Cases
Ovals representing system functions

System Boundary
Box showing system scope

Associations
Lines connecting actors to use cases

Communication Tool

Serves as a bridge between stakeholders, enabling discussion and validation of system scope.

CORE CONCEPTS

Actor and Use Case Concepts

Fundamental Elements of Use-Case Modeling

What is an Actor?

An actor is something outside the system that interacts with the system. An actor may be a person, machine, or other system.

Person	Machine	System

Key Principle: All actors are part of the **system's environment**, not internal components. They exist outside the system boundary and interact with it.

What is a Use Case?

A use case models a dialog between one or more actors and the system. It describes the actions the system takes to deliver something of value to the actor.

Models a Dialog
Represents the conversation between actor and system

Delivers Value
Provides observable and meaningful outcome to actor

Purpose: Describe system behavior from the **external perspective**, focusing on what the system does for its users.

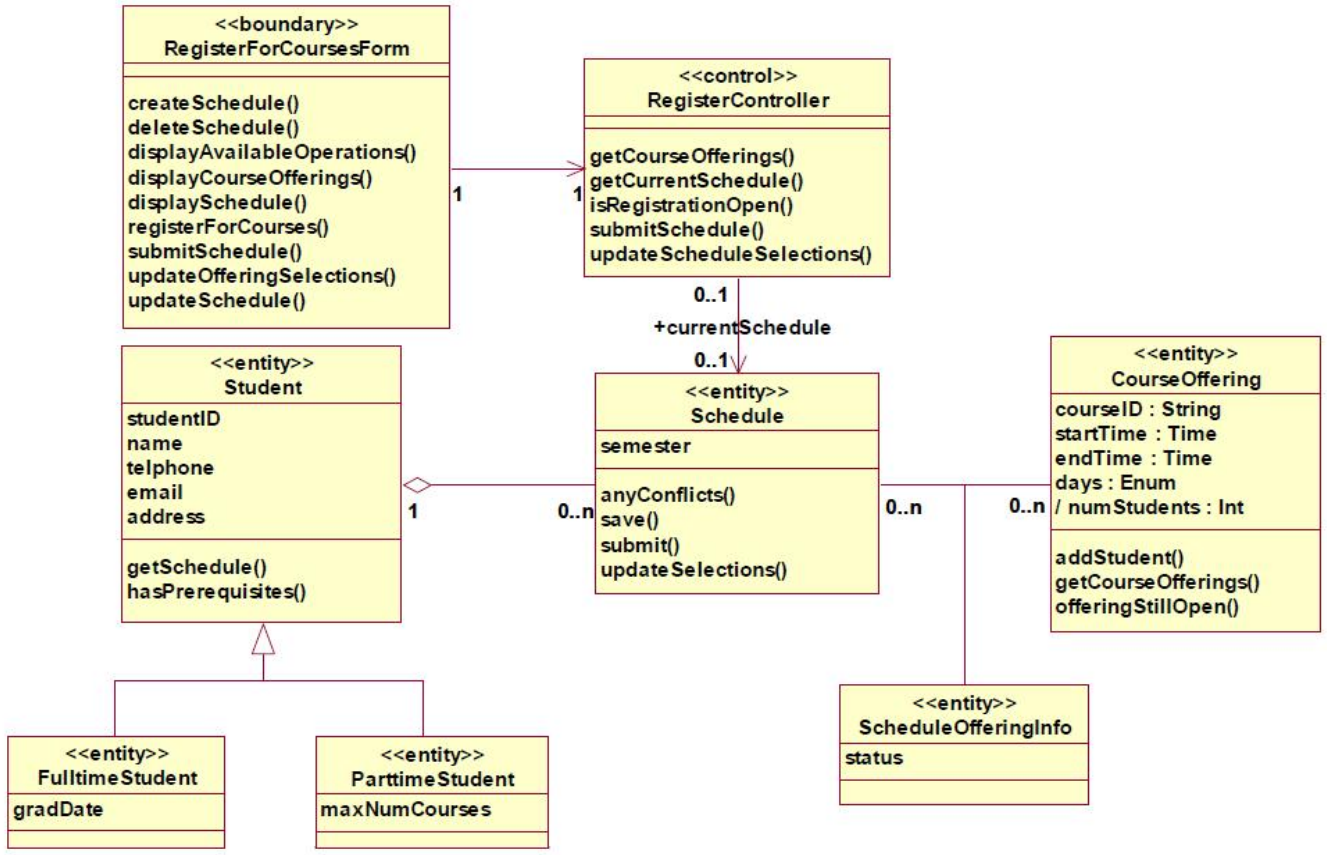
All Actors = System Environment
Actors define the system's context and boundaries

Value Delivery Focus
Use cases emphasize user value over technical implementation

STRUCTURAL MODELING

Class Diagram

Representing System Structure and Relationships



What is a Class Diagram?

A **class diagram** shows the static structure of the system: classes, attributes, operations, and relationships.

Key Components

Classes
Templates for objects (e.g., Sensor, Panel)

Attributes & Func
Data & Func elements of classes

Relationships
Associations, inheritance, composition

BEHAVIORAL MODELING

State Diagram

Modeling System and Object States

What is a State Diagram?

A **state diagram** is a behavioral model that captures the lifecycle of an object or system, showing states, transitions, events, and actions.

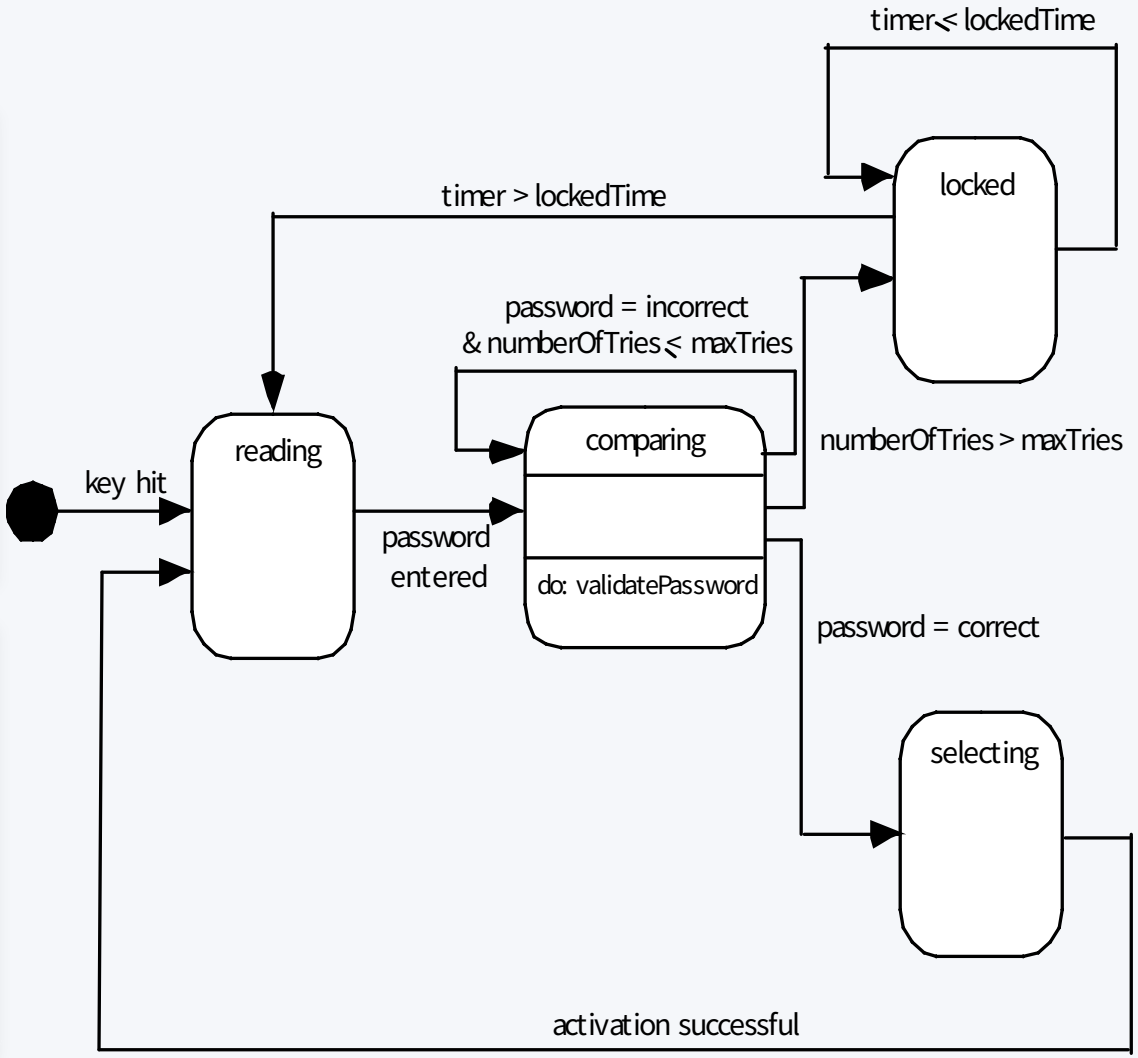
States	Transitions	Events	Actions

Purpose

- Show how objects respond to events
- Model dynamic behavior over time
- Capture lifecycle of critical objects

Key Insight

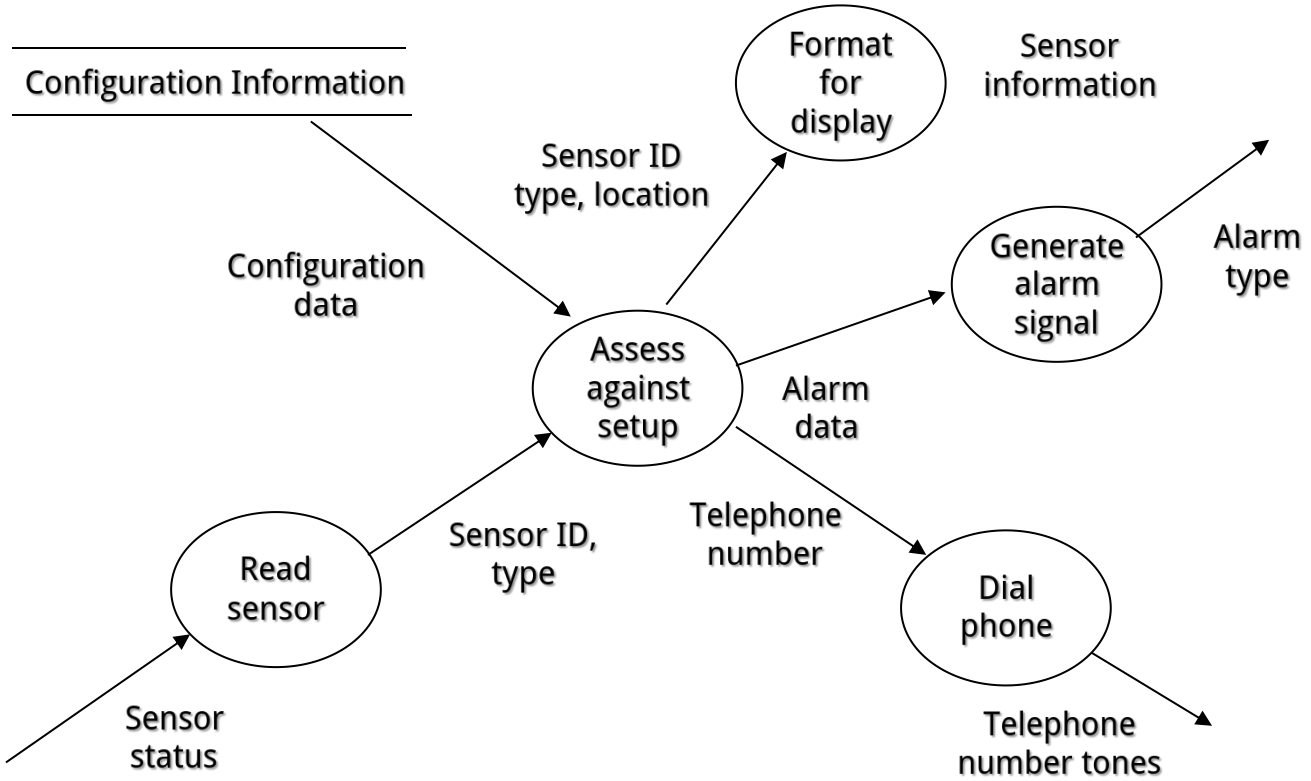
State diagrams reveal **dynamic behavior** that static models (like class diagrams) cannot show, making them essential for understanding system responsiveness.



FLOW MODELING

Data Flow Diagram

Illustrating Data Movement and Transformation



What is a DFD?

Shows **data movement and transformation** through a system, illustrating processing, storage, sources, and sinks.

Key Components

- **Processes**
  Transform data
- **Data Stores**
  Persistent storage
- **External Entities**
  Sources and sinks
- **Data Flows**
  Movement of data

REUSABLE SOLUTIONS

Analysis Patterns

Reusable Solutions for Common Analysis Problems

Pattern name:

A descriptor that captures the essence of the pattern.

Intent:

Describes what the pattern accomplishes or represents

Motivation:

A scenario that illustrates how the pattern can be used to address the problem.

Forces and context:

A description of external issues (forces) that can affect how the pattern is used and also the external issues that will be resolved when the pattern is applied.

Solution:

A description of how the pattern is applied to solve the problem with an emphasis on structural and behavioral issues.

Consequences:

Addresses what happens when the pattern is applied and what trade-offs exist during its application.

Design:

Discusses how the analysis pattern can be achieved through the use of known design patterns.

Known uses:

Examples of uses within actual systems.

Related patterns:

One or more analysis patterns that are related to the named pattern because (1) it is commonly used with the named pattern; (2) it is structurally similar to the named pattern; (3) it is a variation of the named pattern.

TASK 04: NEGOTIATION

Negotiating Requirements

Achieving Win-Win Solutions for All Stakeholders

1 Identify Key Stakeholders

These are the people who will be involved in the negotiation process. Include all parties who have a vested interest.

- End users and customers
- Business management
- Development team leads
- Marketing and sales

2 Determine Win Conditions

Win conditions are not always obvious. Each stakeholder has specific goals and criteria for success.

"What does each stakeholder need to feel this project is successful?"

- Users: Easy to use, fast response
- Management: On time, within budget

3 Negotiate Win-Win

Work toward a set of requirements that lead to "win-win" for all parties—realistic for developers, valuable for customers.

Goal: Balance competing priorities through trade-offs, compromise, and creative solutions.

Balance
Competing priorities

Collaboration
Not confrontation

Agreement
Shared commitment

TASK 06: VALIDATION

Validating Requirements - Part 1

Quality Assurance Questions for Requirements Review

1 Consistent with Objectives?

Is each requirement consistent with the overall objective for the system/product?

2 Proper Abstraction Level?

Have all requirements been specified at the proper level of abstraction?

3 Really Necessary?

Is the requirement really necessary or does it represent an add-on feature that may not be essential?

4 Bounded & Unambiguous?

Is each requirement bounded and unambiguous, leaving no room for multiple interpretations?

5 Have Attribution?

Does each requirement have attribution? Is a source noted for each requirement?

6 Any Conflicts?

Do any requirements conflict with other requirements? Are there logical contradictions?

Validation Goal: Catch requirement defects early when they are cheapest to fix, not during implementation or testing.

VALIDATION

Validating Requirements - Part 2

Technical & Model Quality Checks

7

Achievable in Technical Environment?

Is each requirement achievable in the technical environment that will house the system? Consider hardware, OS, network, and existing infrastructure.

8

Testable Once Implemented?

Is each requirement testable? Can you objectively verify that the requirement has been successfully implemented?

9

Model Reflects System Properly?

Does the requirements model properly reflect the information, function, and behavior of the system to be built?

10

Properly Partitioned?

Has the requirements model been "partitioned" to expose progressively more detailed information about the system?

11

Patterns Used and Validated?

Have requirements patterns been used to simplify the model? Have all patterns been properly validated?



Testability



Feasibility



Completeness

TASK 07: MANAGEMENT

Requirements Management

Identify, Control, and Track Requirements Changes

What is Requirements Management?

A **set of activities** that help the project team identify, control, and track requirements and changes to requirements at any time during the project lifecycle.

> **Continuous Process: Not a one-time activity, but ongoing throughout development and maintenance.**

Feature Traceability

Links features to requirements and implementation, ensuring all features are covered and traceable.

Source Traceability

Identifies the origin of each requirement (stakeholder, regulation, etc.) for impact analysis.

Dependency Traceability

Shows relationships between requirements, highlighting dependencies and potential conflicts.

Subsystem Traceability

Maps requirements to specific subsystems or components, facilitating modular development.

Interface Traceability

Documents interface requirements and how they relate to external systems or components.

Identify
Find and document requirements

Control
Manage changes systematically

Track
Monitor status throughout lifecycle

KNOWLEDGE CHECK

Exercise - Set 1

Test Your Understanding of Requirements Engineering

1. In requirements validation the requirements model is reviewed to ensure its technical feasibility.
    a. True
    b. False
    Answer: b
2. In win-win negotiation, the customer's needs are met even though the developer's need may not be.
    a. True
    b. False
    Answer: b
3. Which of the following is not one of the context-free questions that would be used during project inception?
    a. What will be the economic benefit from a good solution?
    b. Who is against this project?
    c. Who will pay for the work?
    d. Who will use the solution?
    Answer: b
4. The use of traceability tables helps to
    a. debug programs following the detection of run-time errors
    b. determine the performance of algorithm implementations
    c. identify, control, and track requirements changes
    d. none of the above
    Answer: c

KNOWLEDGE CHECK

Exercise - Set 2

Continue Testing Your Understanding

5. The system specification describes the
    a. Function, performance and constraints of a computer-based system
    b. implementation of each allocated system
    c. element software architecture
    d. time required for system simulation
    Answer: a
6. Use-case actors are always people, never system devices.
    a. True
    b. False
    Answer: b
7. Which of the following is not one of the requirement classifications used in Quality Function Deployment (QFD)?
    a. exciting
    b. expected
    c. mandatory
    d. normal
    Answer: c